Laboratory Series No. 04

Dr. Rochdi Boudjehem

2025-11-08

Lab 3 — MPI Fundamentals: Point-to-Point Communication and Collective Operations

This lab introduces distributed-memory parallel programming with MPI (Message Passing Interface). You will implement basic point-to-point communication patterns, collective operations, and analyze the performance impact of communication overhead in distributed systems.

References: Chapter 2 (distributed memory paradigm), MPI programming basics, and performance analysis from Chapter 1.

---

1) Learning Objectives

- Understand MPI execution model and SPMD (Single Program, Multiple Data) paradigm.
- Implement point-to-point communication using `MPI_Send` and `MPI_Recv`.
- Utilize collective operations: `MPI_Bcast`, `MPI_Reduce`, `MPI_Scatter`, `MPI_Gather`.
- Measure and analyze communication overhead vs. computation time.
- Evaluate strong and weak scaling in distributed-memory systems.
- Identify and avoid common MPI pitfalls (deadlocks, message ordering).

---

2) Environment & Setup

Windows (MS-MPI)

1. Install Microsoft MPI (MS-MPI):
   - Download and install MS-MPI Runtime and SDK from Microsoft website
   - Add MS-MPI bin directory to PATH
2. Verify installation:

```
mpiexec --version
```

3. Compile with MinGW-w64:

```
set CC=gcc
%CC% -O3 -march=native -I"C:\Program Files (x86)\Microsoft SDKs\MPI\Include" mpi_lab3.c -o mpi_lab3.exe -L"C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64" -lmsmpi
```

WSL/Linux

```
sudo apt update
sudo apt install build-essential libopenmpi-dev openmpi-bin
mpicc --version
```

Compile and run:

```
mpicc -O3 -march=native -o mpi_lab3 mpi_lab3.c
mpiexec -n 4 ./mpi_lab3 --mode pingpong
```

---

3) Task Description

You will implement a single program with multiple modes:

- **Mode** `hello`: Basic MPI hello world
  - Each process prints its rank and hostname
  - Demonstrates MPI initialization and basic queries
- **Mode** `pingpong`: Ping-pong latency and bandwidth test
  - Process 0 sends messages to Process 1 and receives reply
  - Measure round-trip time for various message sizes
  - Calculate latency (small messages) and bandwidth (large messages)
- **Mode** `ring`: Ring communication pattern
  - Each process sends to next process in ring
  - Last process sends back to first
  - Measure total time for one complete ring traversal
- **Mode** `reduce`: Parallel array reduction
  - Distribute array across processes
  - Each process computes local sum
  - Use `MPI_Reduce` to compute global sum
  - Compare with manual reduction using `MPI_Send`/`MPI_Recv`
- **Mode** `scatter-gather`: Data distribution and collection
  - Root scatters data to all processes
  - Each process performs local computation
  - Root gathers results back
  - Measure scatter/gather overhead vs. computation time

---

4) Starter Code (C/MPI)

Save as `mpi_lab3.c`:

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <mpi.h>

#ifdef _WIN32
#include <windows.h>
static double now_s(){
    LARGE_INTEGER f,c;
    QueryPerformanceFrequency(&f);
    QueryPerformanceCounter(&c);
    return (double)c.QuadPart/(double)f.QuadPart;
}
```

```
#else
#include <time.h>
static double now_s(){
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC,&ts);
    return ts.tv_sec + ts.tv_nsec*1e-9;
}
#endif

static void mode_hello(int rank, int size){
    char hostname[256];
    gethostname(hostname, sizeof(hostname));
    printf("Hello from rank %d of %d on %s\n", rank, size, hostname);
    fflush(stdout);
}

static void mode_pingpong(int rank, int size, int iterations){
    if(size < 2){
        if(rank == 0) printf("pingpong requires at least 2 processes\n");
        return;
    }

    // Test various message sizes (bytes)
    int sizes[] = {8, 64, 512, 4096, 32768, 262144, 2097152};
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    for(int i = 0; i < num_sizes; i++){
        int msg_size = sizes[i];
        char *buffer = (char*) malloc(msg_size);
        if(!buffer) continue;

        MPI_Barrier(MPI_COMM_WORLD);
        double t_start = MPI_Wtime();

        if(rank == 0){
            for(int iter = 0; iter < iterations; iter++){
                MPI_Send(buffer, msg_size, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
                MPI_Recv(buffer, msg_size, MPI_CHAR, 1, 0, MPI_COMM_WORLD, MPI_STA
TUS_IGNORE);
            }
        }
        else if(rank == 1){
            for(int iter = 0; iter < iterations; iter++){
                MPI_Recv(buffer, msg_size, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STA
TUS_IGNORE);
                MPI_Send(buffer, msg_size, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
            }
        }

        double t_end = MPI_Wtime();

        if(rank == 0){
            double elapsed = t_end - t_start;
            double latency_us = (elapsed / (2.0 * iterations)) * 1e6; // one-way l
atency
            double bandwidth_mbps = (2.0 * iterations * msg_size) / (elapsed * 102
```

```
4.0 * 1024.0);
            printf("size=%d bytes, iterations=%d, latency=%.3f us, bandwidth=%.2f
MB/s\n",
                   msg_size, iterations, latency_us, bandwidth_mbps);
        }

        free(buffer);
    }
}

static void mode_ring(int rank, int size, size_t msg_size, int iterations){
    char *buffer = (char*) malloc(msg_size);
    if(!buffer) return;

    int next = (rank + 1) % size;
    int prev = (rank - 1 + size) % size;

    MPI_Barrier(MPI_COMM_WORLD);
    double t_start = MPI_Wtime();

    for(int iter = 0; iter < iterations; iter++){
        if(rank == 0){
            MPI_Send(buffer, msg_size, MPI_CHAR, next, 0, MPI_COMM_WORLD);
            MPI_Recv(buffer, msg_size, MPI_CHAR, prev, 0, MPI_COMM_WORLD, MPI_STAT
US_IGNORE);
        }
        else{
            MPI_Recv(buffer, msg_size, MPI_CHAR, prev, 0, MPI_COMM_WORLD, MPI_STAT
US_IGNORE);
            MPI_Send(buffer, msg_size, MPI_CHAR, next, 0, MPI_COMM_WORLD);
        }
    }

    double t_end = MPI_Wtime();

    if(rank == 0){
        double elapsed = t_end - t_start;
        double time_per_ring = elapsed / iterations;
        printf("ring: processes=%d, msg_size=%zu, iterations=%d, time_per_ring=%.6
f s\n",
               size, msg_size, iterations, time_per_ring);
    }

    free(buffer);
}

static void mode_reduce(int rank, int size, size_t N){
    // Each process gets N/size elements
    size_t local_N = N / size;
    double *local_data = (double*) malloc(local_N * sizeof(double));
    if(!local_data) return;

    // Initialize local data
    for(size_t i = 0; i < local_N; i++){
        local_data[i] = sin((double)(rank * local_N + i)) * 0.5 + 1.0;
    }
```

```
    // Compute local sum
    double local_sum = 0.0;
    double t_comp_start = MPI_Wtime();
    for(size_t i = 0; i < local_N; i++){
        local_sum += local_data[i];
    }
    double t_comp_end = MPI_Wtime();
    double t_comp = t_comp_end - t_comp_start;

    // Method 1: MPI_Reduce
    double global_sum_reduce = 0.0;
    MPI_Barrier(MPI_COMM_WORLD);
    double t_reduce_start = MPI_Wtime();
    MPI_Reduce(&local_sum, &global_sum_reduce, 1, MPI_DOUBLE, MPI_SUM, 0, MPI_COMM
_WORLD);
    double t_reduce_end = MPI_Wtime();
    double t_reduce = t_reduce_end - t_reduce_start;

    // Method 2: Manual reduction using tree
    double global_sum_manual = local_sum;
    MPI_Barrier(MPI_COMM_WORLD);
    double t_manual_start = MPI_Wtime();

    for(int step = 1; step < size; step *= 2){
        if(rank % (2 * step) == 0 && rank + step < size){
            double recv_sum;
            MPI_Recv(&recv_sum, 1, MPI_DOUBLE, rank + step, 0, MPI_COMM_WORLD, MPI
_STATUS_IGNORE);
            global_sum_manual += recv_sum;
        }
        else if(rank % (2 * step) == step){
            MPI_Send(&global_sum_manual, 1, MPI_DOUBLE, rank - step, 0, MPI_COMM_W
ORLD);
            break;
        }
    }

    double t_manual_end = MPI_Wtime();
    double t_manual = t_manual_end - t_manual_start;

    if(rank == 0){
        printf("reduce: N=%zu, processes=%d, local_N=%zu\n", N, size, local_N);
        printf("  computation_time=%.6f s\n", t_comp);
        printf("  MPI_Reduce: sum=%.6f, time=%.6f s\n", global_sum_reduce, t_reduc
e);
        printf("  Manual tree: sum=%.6f, time=%.6f s\n", global_sum_manual, t_manu
al);
        printf("  speedup (manual/MPI_Reduce)=%.2f x\n", t_manual / t_reduce);
    }

    free(local_data);
}

static void mode_scatter_gather(int rank, int size, size_t N){
    size_t local_N = N / size;
```

```
    double *sendbuf = NULL;
    double *recvbuf = (double*) malloc(local_N * sizeof(double));
    double *result = NULL;

    if(!recvbuf) return;

    if(rank == 0){
        sendbuf = (double*) malloc(N * sizeof(double));
        result = (double*) malloc(N * sizeof(double));
        if(!sendbuf || !result) return;

        // Initialize data
        for(size_t i = 0; i < N; i++){
            sendbuf[i] = sin((double)i) * 0.5 + 1.0;
        }
    }

    // Scatter data
    MPI_Barrier(MPI_COMM_WORLD);
    double t_scatter_start = MPI_Wtime();
    MPI_Scatter(sendbuf, local_N, MPI_DOUBLE, recvbuf, local_N, MPI_DOUBLE, 0, MPI
_COMM_WORLD);
    double t_scatter_end = MPI_Wtime();

    // Local computation: square each element
    double t_comp_start = MPI_Wtime();
    for(size_t i = 0; i < local_N; i++){
        recvbuf[i] = recvbuf[i] * recvbuf[i];
    }
    double t_comp_end = MPI_Wtime();

    // Gather results
    double t_gather_start = MPI_Wtime();
    MPI_Gather(recvbuf, local_N, MPI_DOUBLE, result, local_N, MPI_DOUBLE, 0, MPI_C
OMM_WORLD);
    double t_gather_end = MPI_Wtime();

    if(rank == 0){
        double t_scatter = t_scatter_end - t_scatter_start;
        double t_comp = t_comp_end - t_comp_start;
        double t_gather = t_gather_end - t_gather_start;
        double t_comm = t_scatter + t_gather;
        double comm_ratio = t_comm / (t_comm + t_comp);

        printf("scatter-gather: N=%zu, processes=%d, local_N=%zu\n", N, size, loca
l_N);
        printf("  scatter_time=%.6f s\n", t_scatter);
        printf("  computation_time=%.6f s\n", t_comp);
        printf("  gather_time=%.6f s\n", t_gather);
        printf("  total_comm_time=%.6f s (%.1f%% overhead)\n", t_comm, comm_ratio
* 100.0);

        free(sendbuf);
        free(result);
    }
```

```
    free(recvbuf);
}

int main(int argc, char** argv){
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    const char* mode = "hello";
    size_t N = 10000000; // 10M elements
    int iterations = 100;
    size_t msg_size = 1024;

    for(int i = 1; i < argc; i++){
        if(!strcmp(argv[i], "--mode") && i+1 < argc) mode = argv[++i];
        else if(!strcmp(argv[i], "--N") && i+1 < argc) N = (size_t) strtoull(argv[
++i], NULL, 10);
        else if(!strcmp(argv[i], "--iterations") && i+1 < argc) iterations = atoi(
argv[++i]);
        else if(!strcmp(argv[i], "--msg-size") && i+1 < argc) msg_size = (size_t)
strtoull(argv[++i], NULL, 10);
        else if(!strcmp(argv[i], "--help")){
            if(rank == 0){
                printf("Usage: mpiexec -n <procs> %s --mode [hello|pingpong|ring|r
educe|scatter-gather]\n", argv[0]);
                printf("  --N <n>              : array size (for reduce, scatter-g
ather)\n");
                printf("  --iterations <iter>  : iterations (for pingpong, ring)\n
");
                printf("  --msg-size <bytes>   : message size (for ring)\n");
            }
            MPI_Finalize();
            return 0;
        }
    }

    if(strcmp(mode, "hello") == 0){
        mode_hello(rank, size);
    }
    else if(strcmp(mode, "pingpong") == 0){
        mode_pingpong(rank, size, iterations);
    }
    else if(strcmp(mode, "ring") == 0){
        mode_ring(rank, size, msg_size, iterations);
    }
    else if(strcmp(mode, "reduce") == 0){
        mode_reduce(rank, size, N);
    }
    else if(strcmp(mode, "scatter-gather") == 0){
        mode_scatter_gather(rank, size, N);
    }
    else{
        if(rank == 0) fprintf(stderr, "Unknown mode: %s\n", mode);
    }
```

```
    MPI_Finalize();
    return 0;
}
```

---

5) Build Instructions

Windows (cmd.exe with MS-MPI):

```
set CC=gcc
%CC% -O3 -march=native -I"C:\Program Files (x86)\Microsoft SDKs\MPI\Include" mpi_l
ab3.c -o mpi_lab3.exe -L"C:\Program Files (x86)\Microsoft SDKs\MPI\Lib\x64" -lmsmp
i
```

WSL/Linux:

```
mpicc -O3 -march=native -o mpi_lab3 mpi_lab3.c -lm
```

---

6) Run Examples

Windows (cmd.exe):

```
mpiexec -n 4 mpi_lab3.exe --mode hello
mpiexec -n 2 mpi_lab3.exe --mode pingpong --iterations 1000
mpiexec -n 4 mpi_lab3.exe --mode ring --msg-size 1024 --iterations 100
mpiexec -n 4 mpi_lab3.exe --mode reduce --N 100000000
mpiexec -n 4 mpi_lab3.exe --mode scatter-gather --N 10000000
```

WSL/Linux:

```
mpiexec -n 4 ./mpi_lab3 --mode hello
mpiexec -n 2 ./mpi_lab3 --mode pingpong --iterations 1000
mpiexec -n 4 ./mpi_lab3 --mode ring --msg-size 1024 --iterations 100
mpiexec -n 4 ./mpi_lab3 --mode reduce --N 100000000
mpiexec -n 4 ./mpi_lab3 --mode scatter-gather --N 10000000
```

---

7) Data Collection Templates

1. **Ping-Pong Performance Table:**

msg_size (bytes)	iterations	latency (µs)	bandwidth (MB/s)
8	1000		
64	1000		
512	1000		
4096	1000		

2. Strong Scaling (Reduce):

processes	N	local_N	comp_time (s)	comm_time (s)	speedup	efficiency
1	100000000	100000000		0	1.00	1.00
2	100000000	50000000				
4	100000000	25000000				
8	100000000	12500000				

3. Weak Scaling (Scatter-Gather):

processes	local_N	total_N	scatter (s)	comp (s)	gather (s)	comm_overhead (%)
1	10000000	10000000				
2	10000000	20000000				
4	10000000	40000000				
8	10000000	80000000				

---

8) Deliverables

- Completed `mpi_lab3.c` source code.
- A report (02 to 03 pages) including:
  - Description of implementation and communication patterns.
  - Performance data tables filled out.
  - Analysis of performance results (latency, bandwidth, scaling).
  - Discussion of communication overhead impact.
  - A brief summary of observations and conclusions.

---

9) Troubleshooting

- **MPI not found:** Ensure MS-MPI (Windows) or OpenMPI (Linux) is properly installed and in PATH.
- **Deadlock:** Ensure Send/Recv pairs are properly matched. Use `MPI_Sendrecv` for bidirectional communication to avoid deadlocks.
- **Performance variability:** Run multiple times and report best/average. Network congestion can affect results.
- **Large process counts:** If running on single machine, performance may degrade due to resource contention.
- **Windows firewall:** May block MPI communication. Add exception for mpiexec.

Happy distributed computing — you'll see how communication patterns and overhead shape MPI performance!